LAB 11 Firebase Authentication: signup, login and session lifecycle

Unit 4 • 3-hour guided laboratory • CLO-5

Learning goal. Connect an Android Flutter app to a real Firebase project, register users, sign in, sign out and send password-reset email.

Core concepts. Firebase project registration, FlutterFire CLI, generated firebase_options.dart, Auth exceptions, authStateChanges stream, secure identity concepts.

Before you start. Working internet, personal Firebase project and authorized Android device. Cloud authentication requires service-side configuration; source code cannot replace that configuration.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android --org edu.cui.sahiwal mad_lab11
cd mad_lab11
flutter pub add firebase_core firebase_auth
# Install Firebase CLI following the official OS-specific instructions.
# Then run: firebase login
dart pub global activate flutterfire_cli
flutterfire configure
```

Firebase console and CLI prerequisites

1. Create/select a project at https://console.firebase.google.com. In Authentication > Sign-in method enable Email/Password. Avoid any real student passwords in demo accounts; use unique test credentials.
2. Install the Firebase CLI using the method for your operating system (https://firebase.google.com/docs/cli); sign in with firebase login. Install the FlutterFire CLI and ensure Dart's pub global binaries folder is in PATH if flutterfire is not found.
3. Execute flutterfire configure in the Flutter app root and choose Android and the correct Firebase project. This generates lib/firebase_options.dart and updates the relevant Android integration. Run it again after adding Firebase services/plugins as required.
4. A generated firebase_options.dart is project-specific, so it cannot be supplied as a universal copy-paste source file. Both solved app files below are complete once the documented generator has created that configuration.

Solved task 1 | Email/password registration and login with auth gate

Problem and objective. Create one app that supports both signup and login and switches automatically to an authenticated home screen. Do not log or save a plaintext password.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  runApp(const MaterialApp(home: AuthGate()));
}

class AuthGate extends StatelessWidget {
  const AuthGate({super.key});
  @override
  Widget build(BuildContext context) => StreamBuilder<User?>(
    stream: FirebaseAuth.instance.authStateChanges(),
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Scaffold(body: Center(child: CircularProgressIndicator()));
      }
      return snapshot.data == null ? const LoginPage() : const MemberPage();
    },
  );
```

```
}

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});
  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final email = TextEditingController();
  final password = TextEditingController();
  String message = '';
  bool busy = false;
  @override
  void dispose() { email.dispose(); password.dispose(); super.dispose(); }

  Future<void> submit({required bool register}) async {
    if (email.text.trim().isEmpty || password.text.length < 6) {
      setState(() => message = 'Email and 6+ character password required');
      return;
    }
    setState(() => busy = true);
    try {
      if (register) {
        await FirebaseAuth.instance.createUserWithEmailAndPassword(
          email: email.text.trim(), password: password.text);
      } else {
        await FirebaseAuth.instance.signInWithEmailAndPassword(
          email: email.text.trim(), password: password.text);
      }
    } on FirebaseAuthException catch (e) {
      if (mounted) setState(() => message = 'Authentication: ${e.code}');
    } finally {
      if (mounted) setState(() => busy = false);
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Firebase login / signup')),
    body: ListView(padding: const EdgeInsets.all(20), children: [
      TextField(controller: email,
        keyboardType: TextInputType.emailAddress,
        decoration: const InputDecoration(labelText: 'Email')),
      TextField(controller: password, obscureText: true,
        decoration: const InputDecoration(labelText: 'Password')),
      const SizedBox(height: 16),
      FilledButton(onPressed: busy ? null : () => submit(register: false),
        child: const Text('Sign in')),
      OutlinedButton(onPressed: busy ? null : () => submit(register: true),
        child: const Text('Create account')),
      Text(message),
    ]),
  );
}

class MemberPage extends StatelessWidget {
  const MemberPage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Member area')),
    body: Center(child: Column(mainAxisSize: MainAxisSize.min,
      children: [
        Text('Signed in: ${FirebaseAuth.instance.currentUser?.email}'),
        FilledButton(onPressed: () => FirebaseAuth.instance.signOut(),
          child: const Text('Sign out')),
      ],
    )),
  );
}
```

TERMINAL — after FlutterFire configuration

```
flutter pub get
flutter analyze
flutter run
```

How the solution works. Firebase.initializeApp sets the configured backend, and authStateChanges is a stream that emits login/logout state. The UI is gated by that stream rather than a manually stored isLoggedIn boolean. FirebaseAuthException provides codes for duplicate email, invalid credentials, disabled accounts or network failure.

Expected output. Register a test account; the member screen shows the signed-in email. Sign out and sign back in.

Check your work. Try duplicate registration and wrong password; report sanitized error codes rather than exposing secrets.

Solved task 2 | Member profile, email verification and password reset

Problem and objective. Show an authenticated profile and provide logout, verification and reset actions. For a reset demonstration, enter an email address, call the backend and display a non-revealing response.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);
  runApp(const MaterialApp(home: AccountPage()));
}

class AccountPage extends StatefulWidget {
  const AccountPage({super.key});
  @override
  State<AccountPage> createState() => _AccountPageState();
}

class _AccountPageState extends State<AccountPage> {
  final email = TextEditingController();
  String feedback = 'Sign in using Task 1, or provide an email to reset.';
  @override
  void dispose() { email.dispose(); super.dispose(); }

  Future<void> reset() async {
    final value = email.text.trim();
    if (!value.contains('@')) {
      setState(() => feedback = 'Enter a valid email format'); return;
    }
    try {
      await FirebaseAuth.instance.sendPasswordResetEmail(email: value);
      if (mounted) setState(() => feedback =
        'If an account is eligible, a reset email will be sent.');
    } on FirebaseAuthException {
      if (mounted) setState(() => feedback =
        'Reset unavailable; check email settings or try later.');
    }
  }

  Future<void> verify() async {
    final user = FirebaseAuth.instance.currentUser;
    if (user == null) return;
    try {
      await user.sendEmailVerification();
      if (mounted) setState(() => feedback = 'Check your inbox.');
    } on FirebaseAuthException catch (e) {
      if (mounted) setState(() => feedback = 'Verification: ${e.code}');
    }
  }

  @override
```

```
  Widget build(BuildContext context) {
    final user = FirebaseAuth.instance.currentUser;
    return Scaffold(
      appBar: AppBar(title: const Text('Account tools')),
      body: ListView(padding: const EdgeInsets.all(20), children: [
        Text('Current user: ${user?.email ?? 'Not signed in'}'),
        Text('Verified: ${user?.emailVerified ?? false}'),
        TextField(controller: email,
          decoration: const InputDecoration(labelText: 'Reset email')),
        FilledButton(onPressed: reset,
          child: const Text('Send password reset')),
        OutlinedButton(onPressed: user == null ? null : verify,
          child: const Text('Send verification email')),
        OutlinedButton(onPressed: user == null ? null
          : () async { await FirebaseAuth.instance.signOut();
              if (mounted) setState(() {}); },
          child: const Text('Sign out')),
        Text(feedback),
      ]),
    );
  }
}
```

How the solution works. Password resets are performed by Firebase, not by retrieving a stored password. A neutral completion message helps avoid leaking account existence to observers. Verification is only available with a current signed-in user. This exercise demonstrates direct auth API usage; Task 1's auth gate is the preferable way to coordinate navigation after sign-out.

Expected output. A signed-in user can request email verification and sign out. Reset requests can be initiated for a known test email.

Check your work. Check Firebase email templates and that test messages arrive; check the app also handles a network error gracefully.

Security and policy notes

Firebase client config contains application identifiers and is not a server-side secret, but Authentication, Firestore and Storage still need correctly scoped security rules. Email/password credentials belong to the identity provider; never place the password in shared_preferences, source control, application logs or a screenshot. Add account deletion, rate limiting, verification and abuse-monitoring procedures in production.

Scenario-based practice — unsolved

Scenario 1. Clinic appointment identity. Design sign-up, email verification, forgot-password and account-deletion flows without revealing patient data to logged-out users. Identify the backend rule boundaries.

Scenario 2. Shared family phone. A second student opens the app after another student used it. Explain persistent login, sign-out, reauthentication for critical actions and data isolation.

Technical viva — justify your reasoning

1. Why is authStateChanges safer than a local isLoggedIn flag?
2. What is the difference between client-side input validation and Firebase backend authorization?
3. Why should account-existence errors be carefully surfaced in reset flows?
4. Can firebase_options.dart enforce private database access? Why not?
5. Why should authenticated UI be rebuilt after sign-out?
6. What additional security controls would be required for high-value records?

Submission and self-check

Submit registration, login, logout and reset evidence without exposing passwords, plus Firebase test-account and failure-state screenshots.

CSC303 / NOVEMBER 2026 REVISION

LAB 11 EXTENSION | A complete authentication journey

Task 1 demonstrates an auth gate; Task 2 demonstrates account actions in a separate replacement application. For this extension, make Task 2 reachable from the signed-in area of Task 1 rather than launching AccountPage as a stand-alone root.

Integrated navigation and session behavior

• Start the app signed out; show registration/login. Register a designated test account. Use `authStateChanges()` to switch from login to authenticated content.

• Inside authenticated content, navigate to `AccountPage` and expose reset-email and verification actions. After sign-out, the auth gate returns to the signed-out experience.

• Repeat after force-close/relaunch to confirm whether Firebase restores the session; never use a locally stored boolean as the source of truth.

Refresh verification status after returning from email

```
Future<void> refreshVerification() async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;
  try {
    await user.reload();
    if (!mounted) return;
    setState(() {}); // Re-read currentUser in build().
  } on FirebaseAuthException catch (e) {
    if (mounted) setState(() => feedback = e.code);
  }
}
```

Add a `Refresh verification status` button bound to this method. The original display of `emailVerified` can be stale until the current user is refreshed.

Security assertions

• Unauthenticated screens must not display a previous user's private records; navigation hiding is not backend authorization.

• Do not log passwords, private user data or authentication tokens; use reset messages that do not enumerate whether accounts exist.

• Document whether account deletion, reauthentication and abuse controls are excluded from the classroom demo or implemented.

Four acceptance demonstrations

Signed-out startup; successful login and restart; verification refresh; sign-out with private data removed from the UI. Include one invalid-password case and one network failure.